

Compilație Probleme Laborator ASC (x86 AT&T Syntax)

Cuprins

1	Test Laborator 2.1 / 2.2 / 2.3	2
2	Test Laborator 3.1 / 3.2	4
3	Test Laborator 4.1 / 4.2 / 4.3	9

1 Test Laborator 2.1 / 2.2 / 2.3

Întrebarea 1

Ce valoare va reține EAX după executarea următoarei secvențe de instrucțiuni?

```
movl $0, %eax
movb $4, %ah
movb $2, %al
```

Opțiuni: • 258 • 516 (sau 513) • 0 • 1026

Întrebarea 2

Se consideră declarate x: .word 1 și y: .word 2 (sau x: .word 1, y: .word 0, z: .word 2). Ce valoare va avea eax după executarea instrucțiunii mov x, %eax?

Opțiuni: • 2 • 0x00020001 • 1

Întrebarea 3

Fie următoarea declarație în secțiunea .data:

```
str1: .ascii "abc"
str2: .ascii "123" # (sau .ascii "1")
```

Ce se va afișa în urma apelului WRITE următor?

```
movl $4, %eax
movl $1, %ebx
movl $str1, %ecx
movl $5, %edx # (sau $4, %edx)
int $0x80
```

Opțiuni (pentru testul cu "123" și edx=5): • abc • nimic • abc + o valoare reziduală • abc12

Opțiuni (pentru testul cu "1" și edx=4): • abc • nimic • abc + o valoare reziduală • abc1

Întrebarea 4

Fie programul următor. Ce valoare va fi afișată în %eax în urma rulării comenzilor b main, run, stepi, stepi, i r?

```
.data
    x: .long 0x04030201
    y: .long 0x08070605
.text
.global main
main:
    mov x, %eax
    mov y, %al # (sau %ah)
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Listing 1: Cod Sursa (Lab 2.1/2.2)

Opțiuni (pentru mov y, %al): • 0x04030201 • 0x08070605 • 1 • 0x04030501 • 0x04030205

Opțiuni (pentru mov y, %ah): • 0x04030201 • 0x08070605 • 1 • 0x04030205 • 0x04030501

Întrebarea 5

Ce valoare va reține registrul CH după executarea următoarei instrucțiuni?

```
movl $553, %ecx
```

Listing 2: Cod Sursa (Lab 2.3)

Opțiuni: • 10 • 512 • 0 • 2

Întrebarea 6

Fie următoarea declarație în secțiunea .data:

```
str: .ascii "1234"  
x: .byte 97
```

Listing 3: Declarații .data

Ce se va afișa în urma apelului WRITE următor?

```
movl $4, %eax  
movl $1, %ebx  
movl $str, %ecx  
movl $5, %edx
```

Listing 4: Cod Sursa (Lab 2.3)

Opțiuni: • 1234 • 12349 • 123497 • 1234a

Întrebarea 7

Fie următorul program. Precizați secvența corectă de instrucțiuni în debugger, în urma căreia vom obține valoarea 8.

```
.data  
.text  
.global main  
main:  
    movl $8, %eax  
    movl $2048, %ecx  
et_exit:  
    movl $1, %eax  
    movl $0, %ebx  
    int $0x80
```

Listing 5: Cod Sursa (Lab 2.3)

Opțiuni: • b main; run; stepi; stepi; i r cl • b main; run; i r eax • b main; run; stepi; i r ah • b main; run; stepi; stepi; i r ch

2 Test Laborator 3.1 / 3.2

Întrebarea 1

Fie codul de mai jos. Care este valoarea lui s când execuția ajunge la `et_exit`?

```
.data
    s: .long 0
.text
.global main
main:
    mov $1, %edx
    mov $0, %eax
    movl $0xffffffff, %ebx
    divl %ebx
    mov %eax, %ecx

et_loop:
    add %ecx, s
    loop et_loop

et_exit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Listing 6: Cod Sursa (Lab 3.1)

Opțiuni: • 0 • 0xffffffff • 15 • 1

Întrebarea 2

Se stochează în registrul `eax` valoarea `0x40000000`, în `ebx` `0x8`, în `ecx` `0x1` și în `edx` `0x8`. Ce valori vor avea registrii `eax` și `edx` după executarea instrucțiunii `mul %edx`?

Opțiuni: • `eax=32, edx=0` • `eax=32, edx=2` • `eax=2, edx=0` • `eax=0, edx=2`

Întrebarea 3

Se stochează în `%edx` valoarea 0, în `eax` 47 și în `ebx` 15. Ce valori vor avea registrii `eax` și `edx` după executarea instrucțiunii `div %ebx`?

Opțiuni: • `eax=3, edx=0` • `eax=2, edx=0` • `eax=2, edx=3` • `eax=3, edx=2`

Întrebarea 4

Fie codul de mai jos. Care este valoarea depozitată la final în `ecx` (în dreptul etichetei `exit`)?

```
.data
    x: .long 0x80000000
    y: .long 0x70000000
.text
.global main
main:
    mov x, %eax
    cmp y, %eax
    jge label
    mov $5, %ecx
    jmp exit
label:
    mov $6, %ecx
```

```

exit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80

```

Listing 7: Cod Sursa (Lab 3.1)

Opțiuni: • 6 • 5

Întrebarea 5

Fie următorul program. Ce valoare vom obține dacă vom rula cu debuggerul `gdb` `et_exit`, `run`, în `ebx`?

```

.data
.text
.global main
main:
    mov $3, %eax
    shl $2, %eax
    mov $2, %ebx
    mul %ebx
    mov $0, %edx
    mov $8, %ebx
    div %ebx
    sub %eax, %ebx
et_exit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80

```

Opțiuni: • 2 • 3 • 8 • 5

Întrebarea 6

Care este ordinea de trecere prin etichete?

```

.data
.text
.global main
main:
    mov $1, %eax
    mov $2, %ebx
    mov $3, %ecx
    mov $4, %edx
    cmp %ebx, %eax
    je etx
ety:
    cmp %ecx, %edx
    jg etz
    jmp ett
etx:
    jmp ety
etz:
    mov %ebx, %edx
    jmp ety
ett:
    mov $1, %eax
    mov $0, %ebx
    int $0x80

```

Opțiuni: • etx, ety, etz, ety, ett • etx, ety, ett • ety, etx, etz, ett • ety, etz, ety, ett

Întrebarea 7

Fie codul de mai jos. Care sunt valorile lui `eax` și `edx` când execuția ajunge la `label`?

```
.data
    x: .long 17
    y: .long 6
.text
.global main
main:
    mov $1, %edx
    mov x, %eax
    jmp et
    mov $0, %edx
et:
    divl y
label:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Listing 8: Cod Sursa (Lab 3.2)

Opțiuni: • `eax=0x2, edx=0x5` • `eax=0x5, edx=0x2` • `eax=0x3, edx=0x2aaaaaad` • `eax=0x2aaaaaad, edx=0x3`

Întrebarea 8

Se stochează în `EAX` valoarea `0x80000000`, în `EBX` `0x8`, în `ECX` `0x1` și în `EDX` `0x4`. Ce valori vor avea registrele `EAX`, respectiv `EDX` după executarea instrucțiunii `mul %ebx`?

Opțiuni: • `eax=32, edx=0` • `eax=32, edx=4` • `eax=4, edx=0` • `eax=0, edx=4`

Întrebarea 9

Se stochează în `%edx` valoarea `0`, în `eax` `37` și în `ebx` `15`. Ce valori vor avea registrele `eax` și `edx` după executarea instrucțiunii `div %ebx`?

Opțiuni: • `eax=7, edx=2` • `eax=7, edx=0` • `eax=2, edx=0` • `eax=2, edx=7`

Întrebarea 10

Fie codul de mai jos. Care este valoarea depozitată în `z` când ajungem la eticheta `final`?

```
.data
    x: .long 17
    y: .long 6
    x1: .long 5
    y1: .long 9
    z: .space 4
.text
.global main
main:
    mov x, %eax
    mov y, %ebx
    cmp %eax, %ebx
    jge et1
    mov x1, %eax
    cmp %eax, %ebx
    jle et
    mov x, %ebx
et:
```

```

    mov x1, %eax
    mov y1, %ebx
    cmp %eax, %ebx
    jge et2
    add %eax, %ebx
    jmp final
et1:
    add %eax, %ebx
    jmp final
et2:
    sub %eax, %ebx
final:
    mov %ebx, z
    mov $1, %eax
    mov $0, %ebx
    int $0x80

```

Listing 9: Cod Sursa (Lab 3.2)

Opțiuni: • 1 • 12 • 23 • 4

Întrebarea 11

Care este ordinea de trecere prin etichete?

```

.data
.text
.global main
main:
    jmp etb
eto:
    jmp etd
eth:
    jmp eto
etb:
    jmp eth
etd:
    mov $1, %eax
    mov $0, %ebx
    int $0x80

```

Listing 10: Cod Sursa (Lab 3.2)

Opțiuni: • eth, etb, etd, eto • eto, eth, etb, etd • etb, eto, eth, etd • etb, eth, eto, etd

Întrebarea 12

De câte ori se va executa instrucțiunea loop?

```

.data
    x: .long 5
    y: .long 5
    s: .long 0
.text
.global main
main:
    mov x, %ecx
    sub y, %ecx
et:
    add %ecx, s
    loop et
exit:

```

```

mov $1, %eax
mov $0, %ebx
int $0x80

```

Listing 11: Cod Sursa (Lab 3.2)

Opțiuni: • 0x1 • 0x5 • 0xffffffff • este un ciclu infinit • 0x0

Întrebarea 13

Fie următorul program. Ce valoare vom obține dacă vom rula cu debuggerul `gdb` `et_exit`, `run`, în `edx`?

```

.data
.text
.global main
main:
    mov $2, %eax
    mov $3, %ebx
    add %eax, %ebx
    mul %ebx
    mov $0, %edx
    divl $3
    add %eax, %edx
et_exit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80

```

Listing 12: Cod Sursa (Lab 3.2)

Opțiuni: • 0x1 • 0x0 • 0x3 • 0x4

3 Test Laborator 4.1 / 4.2 / 4.3

Întrebarea 1

Ce valori vor fi depozitate în `v` când execuția va ajunge în dreptul etichetei `et_exit`?

```
.data
    v: .space 20  # (sau .space 24)
    n: .long 5
.text
.global main
main:
    lea v, %edi  # (sau movl $v, %edi)
    mov $11, %edx
    mov $0, %ecx
et_loop:
    cmp n, %ecx
    jg et_exit
    mov %edx, # (%edi, %ecx, 4)
    inc %ecx
    inc %edx
    jmp et_loop
et_exit:
    mov $1, %eax
    xor %ebx, %ebx
    int $0x80
```

Listing 13: Cod Sursa (Lab 4.1/4.2/4.3)

Opțiuni: • 11, 12, 13, 14, 15 • 11, 12, ..., 27 • 0, 1, 2, 3, 4, 5 • Execuția nu ajunge la `et_exit` • 11, 12, 13, 14, 15, 16

Întrebarea 2

Ce se va afișa pe ecran? (Notă: codul a fost reasamblat din fragmente)

```
.data
    x: .long 1, 3, 6, 7, 9
    n1: .long 5
    n2: .long 10
    c: .long 0x64636261
    s: .space 11
.text
.global main
main:
    mov $s, %edi
    mov $x, %esi
    movb c, %al
    mov $0, %ecx
et_loop1:
    cmp n2, %ecx
    je et_exit_loop1
    mov %al, (%edi, %ecx, 1)
    inc %ecx
    jmp et_loop1
et_exit_loop1:
    mov $c, %eax
    movb 1(%eax), %al
    mov $0, %ecx
et_loop2:
    cmp n1, %ecx
    je et_exit
```

```

    mov (%esi, %ecx, 4), %ebx
    mov %al, (%edi, %ebx, 1)
    inc %ecx
    jmp et_loop2
et_exit:
    mov $10, %ecx
    movb $0, (%edi, %ecx, 1)
    mov $4, %eax
    mov $1, %ebx
    mov $s, %ecx
    mov $11, %edx
    int $0x80
    mov $1, %eax
    xor %ebx, %ebx
    int $0x80

```

Listing 14: Cod Sursa (Lab 4.1)

Opțiuni: • 61, 61, 61, 61, 61, 61, 61, 61, 61, 61 • aaaaaaaaaa • bbbbbaaaaa • ababaabbab

Întrebarea 3

Ce se va afișa pe ecran?

```

.data
    n: .long 3
    s: .asciz "abc"
.text
.global main
main:
    mov $s, %edi
    mov $0, %ecx
et_loop:
    cmp n, %ecx
    je et_exit
    mov (%edi, %ecx, 1), %al
    sub $'a', %al
    add $'A', %al
    mov %al, (%edi, %ecx, 1)
    inc %ecx
    jmp et_loop
et_exit:
    mov $4, %eax
    mov $1, %ebx
    mov $s, %ecx
    mov $4, %edx
    int $0x80
    mov $1, %eax
    xor %ebx, %ebx
    int $0x80

```

Listing 15: Cod Sursa (Lab 4.1)

Opțiuni: • abc • Abc • Abc + o valoarea reziduala • ABC

Întrebarea 4

Ce se va afișa pe ecran?

```

.data
    n: .long 3
    s: .byte 'a', 'b', 'c'

```

```

    t: .byte 'd', 'e', 'f'
    u: .space 4
.text
.global main
main:
    mov $0, %ecx
et_loop:
    cmp n, %ecx
    je et_exit
    mov $0, %edx
    sub %ecx, %edx
    mov t(, %edx, 1), %al
    mov %al, u(, %ecx, 1)
    inc %ecx
    jmp et_loop
et_exit:
    movb $0, u(, %ecx, 1)
    mov $4, %eax
    mov $1, %ebx
    mov $u, %ecx
    mov $4, %edx
    int $0x80
    mov $1, %eax
    xor %ebx, %ebx
    int $0x80

```

Listing 16: Cod Sursa (Lab 4.1)

Opțiuni: • def • abc • cba • dcb

Întrebarea 5

Fie următorul program. Ce valoare vom obține dacă vom rula `b et_exit, run, ir eax?`

```

.data
n: .long 4
v: .long 0x01020304, 0x05060708, 0x090a0b0c, 0xd0e0f10
.text
.global main
main:
    mov $v, %esi
    mov $1, %ecx
    mov (%esi, %ecx, 4), %eax
    add $4, %esi
    movb (%esi, %ecx, 4), %al
et_exit:
    mov $1, %eax
    xor %ebx, %ebx
    int $0x80

```

Listing 17: Cod Sursa (Lab 4.1)

Opțiuni: • 0x090a0b0c • 0x05060704 • 0x090a0b08 • 0x0506070c

Întrebarea 6

Fie următorul program. Ce valoare vom obține dacă vom rula `b et_exit, run, ir eax?`

```

.data
n: .long 4
v: .long 0x01020304, 0x05060708, 0x090a0b0c, 0xd0e0f10
.text

```

```

.global main
main:
mov $v, %esi
mov $2, %ecx
mov -8(%esi, %ecx, 4), %eax
et_exit:
mov $1, %eax
xor %ebx, %ebx
int $0x80

```

Listing 18: Cod Sursa (Lab 4.1/4.3)

Opțiuni: • 0x090a0b0c • 0x05060708 • Execuție cu eroare • 0x01020304

Întrebarea 7

Ce se va afișa pe ecran?

```

.data
n: .long 9
s: .asciz "a1C95dBx3"
t: .space 10
.text
.global main
main:
mov $s, %esi
mov $t, %edi
mov $0, %ecx
mov $0, %edx
et_loop:
cmp n, %ecx
je et_exit
mov (%esi, %ecx, 1), %al
cmp $'0', %al
jl et2
cmp $'9', %al
jg et2
mov %al, (%edi, %edx, 1)
inc %edx
et2:
inc %ecx
jmp et_loop
et_exit:
movb $0, (%edi, %edx, 1)
inc %edx
mov $4, %eax
mov $1, %ebx
mov $t, %ecx
int $0x80
mov $1, %eax
xor %ebx, %ebx
int $0x80

```

Listing 19: Cod Sursa (Lab 4.2)

Opțiuni: • aCdBx • a1C95dBx3 • 1C95dB3 • 1953

Întrebarea 8

Ce valori vor fi stocate în x când execuția va ajunge la eticheta `et_exit`?

```

.data
x: .long 4, 2, 1, 5, 6
n: .long 5
.text
.global main
main:
mov $x, %esi
mov $0, %eax
et_loop:
cmp n, %eax
je et_exit
mov (%esi, %eax, 4), %ecx
mov $1, %ebx
sal %cl, %ebx
mov %ebx, (%esi, %eax, 4)
inc %eax
jmp et_loop
et_exit:
mov $1, %eax
xor %ebx, %ebx
int $0x80

```

Listing 20: Cod Sursa (Lab 4.2)

Opțiuni: • 0,0,0,0,0 • 16, 4, 1, 25, 36 • 1,2,3,4,5 • 16, 4, 2, 32, 64

Întrebarea 9

Fie următorul program. Ce valori se vor regăsi în registrul %ebx la trecerea prin eticheta **et**?

```

.data
v: .long 15, 21, 30, 16, 18
n: .long 4
.text
.global main
main:
mov $n, %esi
mov $0, %eax
sub n, %eax
mov $0, %ecx
et_loop:
cmp %eax, %ecx
je et_exit
mov (%esi, %ecx, 4), %ebx
et:
dec %ecx
jmp et_loop
et_exit:
mov $1, %eax
xor %ebx, %ebx
int $0x80

```

Listing 21: Cod Sursa (Lab 4.2)

Opțiuni: • 18, 16, 30, 21 • 15, 21, 30, 16 • 15, 21, 30, 16, 18 • 4, 18, 16, 30

Întrebarea 10

Fie următorul program. Ce valoare va avea elementul din mijloc din vector dacă vom rula **b et_exit**, run, x/3xw 8v?

```

.data
v: .long 0x01020304, 0x05060708, 0x090a0b0c
.text
.global main
main:
mov $v, %esi
mov $2, %ecx
mov (%esi, %ecx, 1), %eax
mov %eax, 4(%esi, %ecx, 1)
et_exit:
mov $1, %eax
xor %ebx, %ebx
int $0x80

```

Listing 22: Cod Sursa (Lab 4.2)

Opțiuni: • 0x05060708 • 0x090a0b0c • 0x0506070c • 0x01020708

Întrebarea 11

Ce valoare va fi stocată în s când execuția va ajunge la eticheta `et_exit`?

```

.data
v: .long 15, 21, 30, 16, 18, 12
n: .long 6
s: .long 0
.text
.global main
main:
mov $v, %esi
mov n, %eax
shr $1, %eax
mov $0, %ecx
et_loop:
cmp %eax, %ecx
jge et_exit
mov 0(%esi), %ebx (sau movl (%esi), %ebx)
add %ebx, s (sau addl %ebx, s)
add $8, %esi (sau addl $8, %esi)
inc %ecx
jmp et_loop
et_exit:
mov $1, %eax
xor %ebx, %ebx
int $0x80

```

Listing 23: Cod Sursa (Lab 4.2/4.3)

Opțiuni: • 23 • 129 • 66 • 63

Întrebarea 12

Fie următorul program. Este acesta scris corect? Dacă da, de ce, dacă nu, de ce?

```

.data
x1: .long 5
x2: .long 12
x3: .long 27
n: .long 3
s: .long 0
formatPrintf: .asciz "%d\n"
.text

```

```

.global main
main:
movl $x1, %edi
movl $0, %ecx
et_loop:
cmp n, %ecx
je et_exit
movl (%edi, %ecx, 4), %eax
addl %eax, s
incl %ecx
jmp et_loop
et_exit:
push s
push $formatPrintf
call printf
pop %ebx
pop %ebx
movl $1, %eax
movl $0, %ebx
int $0x80

```

Listing 24: Cod Sursa (Lab 4.3)

Opțiuni: • nu este scris corect, deoarece folosește x1 pe post de array... • nu este scris corect, deoarece n trebuia să fie egal cu 1... • este scris corect, dar nu va funcționa conform așteptărilor... • nu este scris corect, din cauza că intrăm în zone de memorie... • este scris corect, deoarece x1 e doar o adresă, ca numele unui vector

Întrebarea 13

Știind că y este un `.long 0` declarat în secțiunea `.data`, care dintre următoarele poate fi valoarea inițială din x, știind că în urma apelului `printf` se va afișa la `STDOUT` valoarea 1572?

```

main:
movl $0, %ecx
movl $10, %edi
et_while:
cmp x, %ecx
je et_exit
movl x, %eax
movl $0, %edx
div %edi
movl %eax, x
movl %edx, %esi
movl y, %eax
mul %edi
add %esi, %eax
movl %eax, y
jmp et_while
et_exit:
push y
push $formatStr
call printf
popl %ebx
popl %ebx

```

Listing 25: Cod Sursa (Lab 4.3)

Opțiuni: • 1572 • 15720 • 1024 • 83412 • 27510